

VAE

Variational AutoEncoder

논문 완전 정복

Kingma & Welling · 2013 · ICLR

논문 정보

논문	Auto-Encoding Variational Bayes (arXiv: 1312.6114)
저자	Diederik P. Kingma & Max Welling
발표	2013 · ICLR
핵심	Variational Inference + Autoencoder → 확률적 생성 모델
참고	arxiv.org/abs/1312.6114 · angeloyeo.github.io · hyunw.kim/blog

CONTENTS

01	Information Theory 기초	Entropy · Cross-Entropy · KL-Divergence
02	Autoencoder vs VAE	구조 비교 · latent space 차이
03	VAE Architecture	Encoder · Reparameterization · Decoder
04	Reparameterization Trick	왜 필요한가 · 수학적 이유 · 직관
05	MLE & 학습 목표	Maximum Likelihood · intractable 문제
06	ELBO 유도	5단계 수식 전개 · 하한의 의미
07	Loss Function 분해	Reconstruction + KL Loss 구현
08	Python 구현	Keras 코드 라인별 해설

09	VAE 활용 사례	이미지·악물·음악·이상탐지·Diffusion
★	총 정리	전체 흐름 · 핵심 공식 · AE와 차이

01 Information Theory 기초

왜 먼저 알아야 하는가?

VAE의 손실 함수(ELBO)는 KL-Divergence와 Cross-Entropy로 구성된다. 정보량 → 엔트로피 → Cross-Entropy → KL-Divergence 순서로 이해해야 VAE 손실 함수가 왜 그런 형태인지 납득할 수 있다.

① 정보량 (Self-Information)

어떤 사건이 얼마나 "놀라운가"를 수치화한 것이다. 희귀한 사건일수록 정보량이 높고, 확실한 사건은 정보량이 0이다. 두 독립 사건의 정보량을 합산 가능하게 하려면 역수에 log를 씌운 형태가 자연스럽다.

$$I(x) = -\log P(x)$$

Lower $P(x)$ => Higher $I(x)$ $P(x)=1$ => $I(x)=0$ unit: bit(log2) or nat(ln)

② 정보 엔트로피 (Shannon Entropy)

정보량의 기댓값으로, 분포 전체의 평균 불확실성이다. 동전 앞뒤가 각각 0.5일 때 엔트로피 최대, 결과가 확실하면 엔트로피는 0이다.

$$H(X) = E[I(X)] = -\sum_i P(x_i) \log P(x_i)$$

Uniform distribution => max entropy Deterministic distribution => entropy = 0

③ Cross-Entropy

실제 분포 P와 예측 분포 Q의 차이를 측정한다. H(P)가 상수라면 CE 최소화 = KL 최소화와 동일하다. 딥러닝 분류 모델의 loss로 쓰이는 이유가 여기에 있다.

$$CE(P, Q) = -\sum_x P(x) \log Q(x) = H(P) + D_{KL}(P \parallel Q)$$

P: true Q: predicted $CE \geq H(P)$ CE minimization = KL minimization

④ KL-Divergence

Cross-Entropy에서 Entropy를 뺀 값으로, 두 분포 사이의 순수한 차이만 측정한다. "P를 따르는 사람이 Q라 가정했을 때 추가로 필요한 비트 수"로 직관적으로 이해할 수 있다.

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log P(x)/Q(x) \geq 0$$

$P=Q \Rightarrow D_{KL}=0$ Asymmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P) \Rightarrow$ called "divergence"

KL-Divergence 핵심 성질

Non-negative	$D_{KL} \geq 0$ 항상. Jensen's Inequality: $-\log$ 는 convex이므로 $E[-\log(Q/P)] \geq -\log(E[Q/P]) = 0$.
비대칭성	$D_{KL}(P Q) \neq D_{KL}(Q P) \rightarrow$ 거리가 아닌 "발산". 대칭 필요시 Jensen-Shannon Divergence 사용.
CE 관계	$CE(P,Q) = H(P) + D_{KL}(P Q)$. $H(P)$ 상수이면 CE 최소화 = KL 최소화. supervised learning의 본질은 두 분포의 KL을 줄이는 것이다.
VAE 역할	Encoder 출력 $q_\phi(z x)$ 를 표준정규분포 $p(z)=N(0,1)$ 에 가깝게 강제. KL Loss 최소화 $\rightarrow$ latent space 구조화 $\rightarrow$ 새 데이터 생성 가능.

02 Autoencoder vs VAE

Autoencoder (AE)

입력을 저차원 latent space로 압축했다가 복원하는 신경망. hidden layer 뉴런 수를 줄이거나 노이즈를 주입하는 제약으로 핵심 feature를 추출한다. Encoder가 핵심이며, 새로운 데이터 생성은 불가능하다 — latent space가 비구조적이기 때문이다.

Variational Autoencoder (VAE)

latent vector를 하나의 점이 아닌 확률 분포 파라미터(μ , σ)로 표현한다. latent space 전체를 $N(0, I)$ 형태로 강제하여 연속적이고 구조화된 공간을 만든다. Decoder가 핵심이며 새로운 데이터 생성이 주목표인 Generative Model이다.

구분	Autoencoder (AE)	Variational AE (VAE)
목적	Encoder — 압축 & 복원	Decoder — 새 데이터 생성
latent z	결정론적 벡터 (점 하나)	확률 분포 파라미터 (μ , σ)
latent space	임의적 · 불연속적	$N(0, I)$ 에 근사 · 연속적
새 데이터	생성 불가	z 샘플링으로 무한 생성
학습 목표	Reconstruction Error 최소화	ELBO 최대화 (Recon + KL)

AE의 latent space는 구조가 없어 임의의 z를 넣으면 엉뚱한 이미지가 나온다. VAE는 KL Loss로 latent space를 $N(0, I)$ 에 가깝게 강제하여 이 문제를 해결한다.

03 VAE Architecture

3개 파트 완전 이해

ENCODER $q_{\phi}(z x)$	Input $x \rightarrow$ 가우시안 분포 파라미터 μ, σ 출력 · 이미지 픽셀이 가우시안 분포를 따른다고 가정, μ head / σ head 두 개 출력 · $q_{\phi}(z x) \sim N(\mu_{\phi}(x), \sigma_{\phi}(x)^2 I)$ — 파라미터 ϕ 로 정의된 근사 사후 분포
REPARAM $z = \mu + \sigma \cdot \epsilon$	$\epsilon \sim N(0, I)$ 샘플링 후 $z = \mu + \sigma \cdot \epsilon$ · 확률적 sampling을 결정론적 연산으로 변환 $\rightarrow$ Backprop 가능 · ϵ 은 외부 상수로 취급. gradient는 μ, σ 를 통해서만 흐름
DECODER $p_{\theta}(x z)$	$z \rightarrow$ 원본 x 복원 ($p_{\theta}(x z)$ 추정) · MLE 방식으로 z 가 주어졌을 때 x 가 나올 확률이 최대인 분포를 학습 · 이미지(0~1 픽셀): Bernoulli 가정 $\rightarrow$ Binary CE / 연속값: Gaussian 가정 $\rightarrow$ MSE

수식 정리

Encoder	$q_{\phi}(z x) \sim N(\mu_{\phi}(x), \sigma_{\phi}(x)^2 I)$ — x 를 받아 가우시안 파라미터 출력
z 샘플링	$z = \mu + \sigma \cdot \epsilon, \epsilon \sim N(0, I)$ — Reparameterization Trick
Decoder	$p_{\theta}(x z) \sim \text{Bernoulli}(f_{\theta}(z))$ 또는 $N(\mu_{\theta}(z), \sigma^2 I)$
Prior	$p(z) = N(0, I)$ — latent z 는 표준정규분포를 따른다고 가정
학습 목표	$\log p_{\theta}(x)$ 최대화 = ELBO 최대화 (intractable $\rightarrow$ ELBO로 우회)

04 Reparameterization Trick

문제: 직접 Sampling은 미분 불가능

VAE 학습에는 Encoder ϕ 까지 gradient 역전파가 필요하다. z 를 직접 샘플링하면 확률적 연산이라 $\partial z / \partial \mu$ 계산 자체가 불가능해진다.

$$z \sim \mathcal{N}(\mu, \sigma^2) \implies \partial z / \partial \mu = ??$$

Stochastic sampling => gradient cannot flow through => Encoder cannot be trained x

해결: 확률성을 ϵ 으로 분리

랜덤성을 외부 상수 ϵ 으로 분리하면 z 는 μ, σ 에 대한 결정론적 함수가 된다. 덧셈·곱셈은 미분 가능하므로 gradient가 끊기지 않고 Encoder까지 흐른다.

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \implies \partial z / \partial \mu = 1, \quad \partial z / \partial \sigma = \epsilon$$

Deterministic function of mu, sigma epsilon = external constant chain rule intact v

구분	직접 Sampling	Reparameterization
z 계산	$z \sim \mathcal{N}(\mu, \sigma^2)$ 확률적	$z = \mu + \sigma \cdot \epsilon$ 결정론적
$\partial z / \partial \mu$	계산 불가	= 1
$\partial z / \partial \sigma$	계산 불가	= ϵ
backprop	Encoder 전파 안됨	Encoder까지 완전 전파
랜덤성	z 자체가 확률	ϵ 이 담당 (외부 상수)

사용하는 두 가지 이유

① Backprop (본질)

z 를 $\mu + \sigma \cdot \epsilon$ 사칙연산으로 표현 $\rightarrow$ chain rule 유지 $\rightarrow$ Encoder(ϕ)까지 gradient 전파.
이것이 없으면 VAE 학습 자체가 불가능하다.

② 다양성 (부수)

동일한 x 에도 ϵ 이 매번 달라져 z 가 분포 위의 다른 점을 샘플링 $\rightarrow$ 매번 다른 output.
AE와 달리 진정한 생성 모델로 동작.

05 MLE & 학습 목표

Maximum Likelihood Estimation

"관찰된 데이터가 나올 확률을 가장 크게 만드는 파라미터를 찾는 방법"이다. 곱셈 형태의 Likelihood는 수치 underflow 문제가 있으므로 log를 취해 합으로 변환한다.

$$\theta^* = \arg \max_{\theta} \log p_{\theta}(x) = \arg \max_{\theta} \sum_i \log p_{\theta}(x_i)$$

Likelihood -> Log-Likelihood product -> sum numerical stability

직관적 예시 — 동전 10번 중 앞면 7번

MLE 질문	"이 결과(7앞 3뒤)가 나올 확률을 가장 크게 만드는 p는?"
p=0.5	$L=(0.5)^7 \times (0.5)^3 = 0.000977 \rightarrow$ 낮음
p=0.7	$L=(0.7)^7 \times (0.3)^3 = 0.002222 \rightarrow$ 최대 $\rightarrow$ MLE 추정값
p=0.9	$L=(0.9)^7 \times (0.1)^3 = 0.000478 \rightarrow$ 낮음
VAE 연결	x(이미지)가 주어졌을 때 $p_{\theta}(x z)$ 를 최대화하는 θ 를 학습 — 동일한 원리

왜 직접 계산이 불가능한가?

$p_{\theta}(x) = \int p_{\theta}(x|z) p(z) dz$ 는 연속 고차원 z에 대한 적분이라 계산 불가능(intractable)하다. 이 문제를 해결하기 위해 ELBO를 도입한다.

$$p_{\theta}(x) = \int p_{\theta}(x|z) p(z) dz \quad \rightarrow \quad \text{intractable}$$

Integration over continuous high-dim z => cannot compute => use ELBO instead

06 ELBO 유도 5단계

ELBO란?

계산 불가능한 $\log p_\theta(x)$ 의 계산 가능한 하한(Lower Bound)이다. $\log p_\theta(x) \geq \text{ELBO}$ 가 항상 성립하므로 ELBO를 최대화하면 $\log p_\theta(x)$ 도 올라가는 방향으로 학습이 진행된다.

STEP 1 1을 곱하는 트릭	$\int q_\phi(z x) dz = 1$ 이므로 양변에 곱해도 값 불변 $\rightarrow \log p_\theta(x) = \int q_\phi(z x) \cdot \log p_\theta(x) dz$ (적분 안으로 이동)
STEP 2 Bayes Rule 적용	$p_\theta(x) = p_\theta(x z) \cdot p(z) / p_\theta(z x) \rightarrow \log p_\theta(x) = \int q_\phi \cdot \log[p_\theta(x z) \cdot p(z) / p_\theta(z x)] dz$
STEP 3 $q_\phi(z x) / q_\phi(z x)$ 삽입	= 1이므로 값 변화 없음. 나중에 KL-divergence 형태로 묶기 위한 준비
STEP 4 log 분리 — 3개 항	$= \int q_\phi \cdot \log p_\theta(x z) dz \leftarrow \text{Reconstruction 기댓값} + \int q_\phi \cdot \log[p(z)/q_\phi] dz \leftarrow \text{Regularization 항} + \int q_\phi \cdot \log[q_\phi/p_\theta(z x)] dz \leftarrow \text{계산 불가 항}$
STEP 5 KL 형태로 정리 + ELBO 추출	$= E_q[\log p_\theta(x z)] - D_{KL}(q_\phi(z x) \parallel p(z)) + D_{KL}(q_\phi \parallel p_\theta(z x)) \rightarrow \text{세 번째 항 } p_\theta(z x) \text{는 계산 불가. } KL \geq 0 \text{ 이므로 제거} \rightarrow \text{부등호로 ELBO 표현}$

ELBO 최종 형태

$$\log p_\theta(x) \geq E_q[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel p(z)) \equiv \mathcal{L}_{\text{ELBO}}$$

$KL \geq 0 \Rightarrow \text{removing intractable term gives lower bound} \Rightarrow \text{maximize ELBO}$

ELBO를 최대화하면 $\log p_\theta(x)$ 도 올라간다. $p_\theta(z|x)$ 를 알 수 없어도 $KL \geq 0$ 성질로 학습을 진행할 수 있다.

07 Loss Function 완전 분해

$$\mathcal{L} = \mathcal{L}_{rec} + \mathcal{L}_{KL} \quad \longrightarrow \quad \text{Minimize} \equiv -\mathcal{L}_{ELBO}$$

Reconstruction quality + latent space regularization -> trained simultaneously

① Reconstruction Loss

Decoder가 latent z 로 원본 x 를 얼마나 잘 복원하는지 측정한다. 모든 z 에 대한 적분이 불가능하므로 Monte-Carlo 시뮬레이션으로 근사하며, 실제로는 $L=1$ (단 1회 샘플링)만으로도 잘 동작한다.

$$\mathcal{L}_{rec} = E_{q_{\phi}(z|x)}[-\log p_{\theta}(x|z)]$$

Bernoulli decoder => Binary Cross-Entropy Gaussian decoder => MSE

Reconstruction Loss 상세

Monte-Carlo	$L=1$ 샘플링으로 근사. $\int q_{\phi}(z x) \cdot \log p_{\theta}(x z) dz$ 계산
Bernoulli	$p_{\theta}(x z) = \prod f_i^{x_i} \cdot (1-f_i)^{(1-x_i)} \rightarrow$ Binary Cross-Entropy
Gaussian	$p_{\theta}(x z) = N(x; \mu_{\theta}(z), \sigma^2) \rightarrow$ MSE(L2 Loss)

② KL Loss (Regularization)

Encoder 출력 $q_{\phi}(z|x)$ 가 사전 분포 $p(z)=N(0,1)$ 에 얼마나 가까운지 측정한다. Encoder와 Prior 모두 Gaussian으로 가정하면 closed-form으로 계산 가능하다.

$$\mathcal{L}_{KL} = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

Closed-form (Gaussian prior + Gaussian posterior) no integration needed

두 Loss 비교

Recon 역할	z로 x를 잘 복원하는가 → 생성 품질
KL 역할	latent가 $N(0, I)$ 에 가까운가 → latent space 구조화
Recon 구현	Binary CE (Bernoulli) 또는 MSE (Gaussian)
KL 구현	$-\frac{1}{2} \sum (\log \sigma^2 + \mu^2 / \sigma^2) \rightarrow$ closed-form, 적분 불필요
β -VAE	KL에 $\beta > 1$ 가중 → 각 latent 차원이 독립 의미 (disentangled)

08 Python 구현 (Keras)

vae_keras.py

```
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder

        # ① Loss Tracker 세 개 등록
        self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = keras.metrics.Mean(name="reconstruction_loss")
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")

    def train_step(self, data):
        with tf.GradientTape() as tape:
            # ② Encoder:  $x \rightarrow (z\_mean, z\_log\_var, z)$ 
            z_mean, z_log_var, z = self.encoder(data)

            # ③ Decoder:  $z \rightarrow reconstruction$ 
            reconstruction = self.decoder(z)

            # ④ Reconstruction Loss (Binary Cross-Entropy)
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(
                    keras.losses.binary_crossentropy(data, reconstruction),
                    axis=(1, 2)))

            # ⑤ KL Loss =  $-\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$ 
            kl_loss = -0.5 * (1 + z_log_var
                              - tf.square(z_mean)
                              - tf.exp(z_log_var))
            kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))

            # ⑥ Total Loss = Reconstruction + KL
            total_loss = reconstruction_loss + kl_loss

            # ⑦ Gradient 계산 및 파라미터 업데이트
            grads = tape.gradient(total_loss, self.trainable_weights)
            self.optimizer.apply_gradients(
                zip(grads, self.trainable_weights))
```

코드 핵심 해설

② encoder 출력	$z_mean(\mu)$, $z_log_var(\log \sigma^2)$, z (reparameterization 적용 샘플)
④ Recon Loss	<code>binary_crossentropy</code> → Bernoulli 가정. <code>axis=(1,2)</code> 로 H×W 픽셀 합산.

⑤ KL closed-form

$z_log_var = \log \sigma^2$, $\exp(z_log_var) = \sigma^2$. $-\frac{1}{2} \cdot \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2) = KL(N(\mu, \sigma^2) || N(0, I))$
closed-form.

⑦ GradientTape

Encoder(ϕ)와 Decoder(θ) 파라미터를 동시 업데이트. Reparameterization 덕분에 Encoder까지 gradient 전파.

09 VAE 활용 사례

VAE는 연속적이고 구조화된 latent space 덕분에 다양한 분야에 활용된다.

이미지 생성 & 편집

· CelebA 얼굴 데이터 학습 → 존재하지 않는 새 얼굴 생성 · latent space에서 z_1 , z_2 를 선형 보간 → 자연스러운 변환 (AE로는 불가) · 머리카락 색, 안경 등 속성을 latent vector 조작으로 제어 · β -VAE: 각 latent 차원이 독립적 의미를 갖도록 강제 (disentangled)

약물 분자 설계

· 분자 구조를 연속 latent space에 임베딩 → 유사 분자 생성 · "Automatic Chemical Design Using a Data-Driven Continuous Representation" (2018) · 신약 개발 후보 분자 탐색 시간 획기적 단축 · latent space gradient descent로 원하는 분자 특성 최적화

음악 & 텍스트 생성

· MusicVAE: 음악 구절을 latent space에 임베딩 → 두 멜로디 보간 · 텍스트 VAE: 문장을 연속 벡터로 → 문체 전환, 감성 조절 · Conditional VAE(CVAE): 조건을 입력받아 원하는 스타일 생성

이상 탐지

· 정상 데이터만 학습 → 비정상은 reconstruction error가 높아짐 · 제조업 불량 검출, 의료 영상 병변 탐지 (MRI, CT) · Unsupervised 학습으로 레이블 없이 이상 탐지 가능

Stable Diffusion의 기반

· Latent Diffusion = VAE(압축) + U-Net(diffusion) + CLIP(조건) · Stable Diffusion은 VAE로 이미지를 latent로 압축 후 그 위에서 diffusion 수행 · DALL-E, Imagen 등 최신 생성 모델 모두 VAE 또는 그 변형을 핵심으로 사용 · VAE를 이해하면 최신 생성 AI 구조를 전부 파악할 수 있다

VAE는 GAN과 달리 mode collapse나 불안정 학습 없이 수렴한다. Stable Diffusion을 비롯한 최신 Latent Diffusion Model의 핵심 구성 요소이다.



총 정리

VAE 전체 흐름

INPUT x	학습 이미지 — 고차원 픽셀 행렬
ENCODER q_ϕ	$x \rightarrow \mu, \sigma$ 출력. 이미지를 가우시안 분포 파라미터로 변환
REPARAMETERIZE	$\epsilon \sim N(0, I)$ 샘플 $\rightarrow z = \mu + \sigma \cdot \epsilon$ (Backprop 가능하게 만드는 핵심)
LATENT z	압축된 잠재 표현. KL Loss로 $N(0, I)$ 에 가깝도록 강제
DECODER p_θ	$z \rightarrow x'$ 생성. MLE 기반으로 $p_\theta(x z)$ 학습
LOSS	$L_{rec} \text{ (CE/MSE)} + L_{KL} = \text{Total Loss} \rightarrow \text{Minimize}$
BACKPROP	Gradient $\rightarrow$ Decoder(θ) + Encoder(ϕ) 동시 업데이트

핵심 공식 한눈에

Self-Info	$I(x) = -\log P(x)$
Entropy	$H(X) = -\sum P(x_i) \log P(x_i)$
KL-Divergence	$D_{KL}(P\ Q) = \sum P(x) \log P(x)/Q(x) \geq 0$
Reparameterize	$z = \mu + \sigma \cdot \epsilon, \epsilon \sim \mathcal{N}(0, I)$
ELBO	$\mathcal{L}_{ELBO} = E_q[\log p_\theta(x z)] - D_{KL}(q_\phi\ p(z))$
KL closed-form	$\mathcal{L}_{KL} = -1/2 \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$
Total Loss	$\mathcal{L} = \mathcal{L}_{rec} + \mathcal{L}_{KL} \rightarrow \text{Minimize}$

AE와의 결정적 차이

구분	Autoencoder	VAE
----	-------------	-----

latent 표현	결정론적 점 (벡터)	확률 분포 파라미터 (μ, σ)
새 데이터	생성 불가	z 샘플링으로 무한 생성
latent space	비구조적	$N(0, I)$ 강제 · 구조적

$$\text{VAE} = \text{Encoder}(\mu, \sigma) + \text{Reparameterization}(\epsilon) + \text{Decoder} + \text{ELBO Loss}$$

Reparameterization Trick → Backprop 가능

ELBO → intractable MLE를 우회

KL Loss → latent space를 정규분포로 유지

Reconstruction Loss → 생성 품질 보장